

Multi-Tenant AI Customer Support & Commerce SaaS Platform

Complete Architectural Blueprint & Technical Implementation Master Plan

DOCUMENT PURPOSE Complete System Architecture & MVP Engineering Blueprint	PRIMARY AI STRATEGY Local AI (Ollama) / Provider-Agnostic Core
PRIMARY TECH STACK Next.js, TypeScript, FastAPI, PostgreSQL + pgvector, Redis, Celery	KEY DIFFERENTIATORS WooCommerce Integration, Action Tools, WhatsApp Handoff

Based on comprehensive requirements analysis, this product is defined as a **Multi-Tenant AI Customer Support & Commerce SaaS Platform** rather than simply a chatbot. The most important architectural decision for the MVP is that **no paid AI API is required initially**. The platform runs AI locally using Ollama and a local open-source LLM, while maintaining a provider-agnostic architecture so OpenAI, Gemini, Claude, and other external APIs can be integrated seamlessly in later phases.

1. Product Definition

Working Product Name: AI Customer & Commerce Assistant SaaS

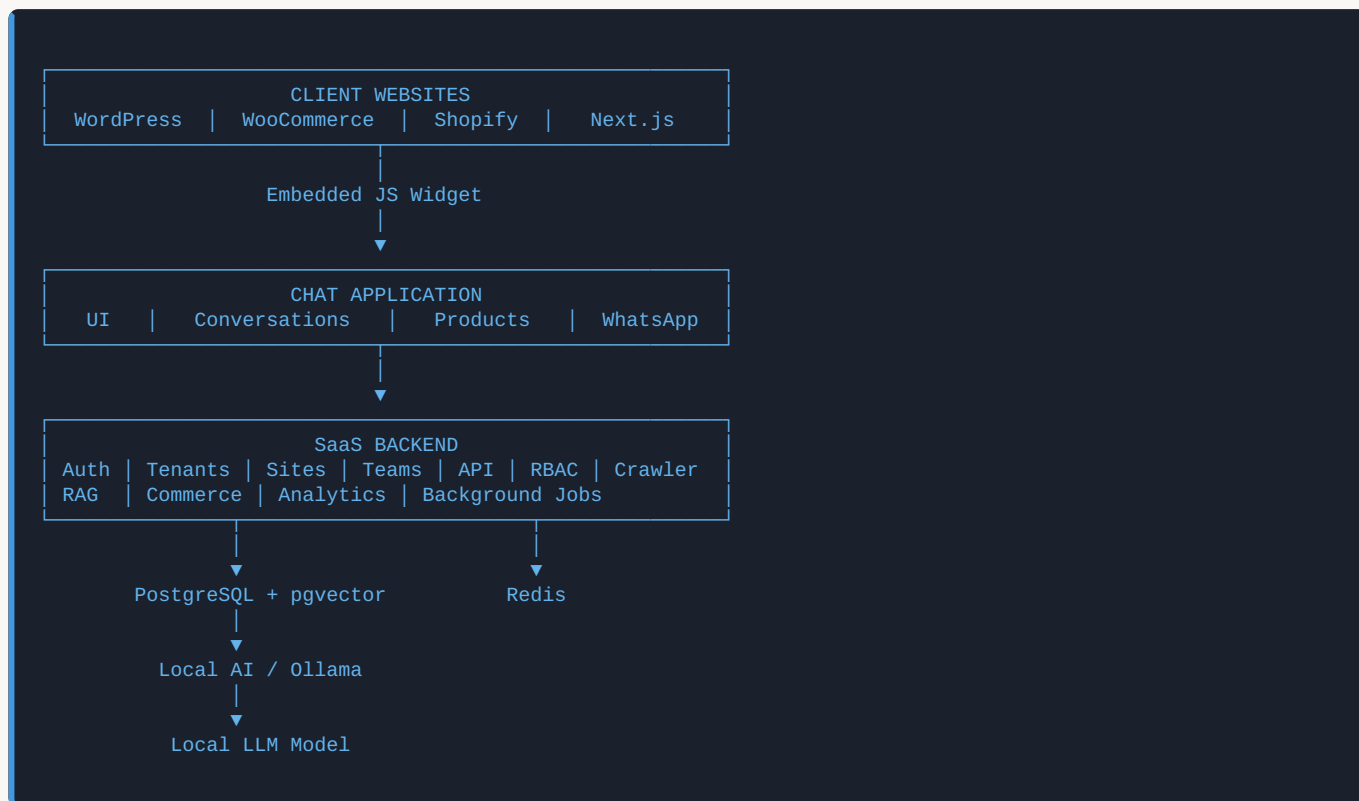
Core Concept: A business owner signs up → creates a website/project → enters their website URL → the platform automatically discovers and reads the website → creates a knowledge base → deploys an AI chatbot → business owner embeds the chatbot into their website with a single JavaScript script tag.

Core Capabilities & Feature Matrix

- Answer questions strictly derived from website content
- Search website information and read pages automatically
- Parse and respect sitemap.xml, sitemap_index.xml, and robots.txt rules
- Crawl WordPress websites and utilize WordPress REST API where appropriate
- Understand WooCommerce products: display product cards, search/filter products, add products to cart, redirect users to checkout
- Future support for Shopify commerce operations
- Intelligently detect non-resolvable inquiries and offer seamless WhatsApp human support handoff
- Maintain continuous conversation histories, visitor sessions, and state
- Support multi-tenant organizations, team management, granular RBAC, and multi-website monitoring
- Support white-label branding customization (colors, logos, avatars, custom instruction prompts)
- Provide real-time analytics, conversation resolution metrics, and conversion tracking
- Execute AI locally via Ollama with full architectural support for external AI APIs

2. Product Architecture

The system is structured across four distinct operational layers:



3. Recommended Technology Stack

Frontend Architecture

The SaaS dashboard and administration portal are engineered as a modern, high-performance Next.js application utilizing:

- **Framework & Language:** Next.js (App Router), React, TypeScript
- **Styling & Components:** Tailwind CSS, shadcn/ui
- **State & Form Management:** TanStack Query (React Query), React Hook Form, Zod (Schema Validation)
- **Data Visualization:** Recharts

Backend Architecture

The core API, orchestration engine, crawler, and RAG pipelines utilize **Python and FastAPI**. Python is specifically chosen due to its native superiority in web crawling, HTML processing, AI pipeline handling, embedding generation, vector search integration, and background worker queues.

- **Core Tech:** Python, FastAPI, Pydantic, SQLAlchemy, Alembic
- **Task Processing & Caching:** Celery, Redis
- **Crawling & Processing:** BeautifulSoup4, httpx, Playwright

4. Database Architecture & Schema Strategy

The platform relies on **PostgreSQL with the pgvector extension** enabled as its primary database solution. PostgreSQL handles relational data alongside vector embeddings within a unified, highly reliable system, eliminating the cost and operational complexity of an external vector database during the MVP phase.

```

PostgreSQL (pgvector Enabled)
├── Users / Organizations / Organization Members
├── Websites / Website Settings / Website Domains
├── Teams / Roles / Permissions
├── Conversations / Messages / Visitors
├── Products / Product Cache / Cart Sessions
├── Crawl Jobs / Crawl Pages
├── Knowledge Documents / Knowledge Chunks (with Vector Embeddings)
├── Integrations / WhatsApp Settings
└── Audit Logs / Analytics Events / API Keys

```

5. Caching & Background Task Broker (Redis)

Redis functions as the central event broker and transient data layer for:

- Celery task broker and backend results storage
- Web crawling state, queue management, and temporary document buffers
- API rate limiting, session caching, and request queues
- Real-time event handling and streaming response support

6. Local AI Architecture & Provider Abstraction

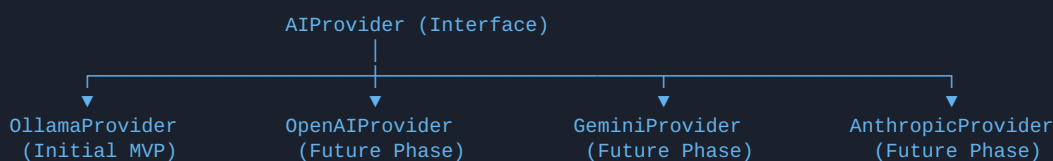
To eliminate initial API reliance, the platform decouples the core business logic from specific AI vendors using a robust provider pattern abstraction:

```

Chat Request → FastAPI → Retriever → PostgreSQL + pgvector → Relevant Chunks
Response ← Local LLM ← Ollama ← Prompt Construction ← Relevant Chunks

```

The software interface is built around an extensible AIProvider class hierarchy:



7. Website Crawling & Discovery Engine

The crawling pipeline follows a structured, robust protocol designed for high fidelity and policy compliance:

1. **Domain Normalization & Validation:** Validate target host and protocols.
2. **Robots Policy Check:** Fetch and parse `/robots.txt` to identify crawl rules and sitemap directives.
3. **Sitemap Discovery:** Check `/sitemap.xml` and `/sitemap_index.xml`. Parse published links.
4. **Controlled Fallback Crawl:** If sitemaps are absent, discover pages via controlled homepage internal link crawling, enforcing depth limits, domain restrictions, maximum page caps, and SSRF protections.
5. **Content Extraction & Cleaning:** Extract titles, headings, structural paragraphs, tables, lists, and structured schema metadata using BeautifulSoup4. Remove repetitive navigation/footer noise.

6. **Document Chunking & Vectorization:** Segment text into configured chunks, generate embeddings locally, and store vector representations in pgvector.

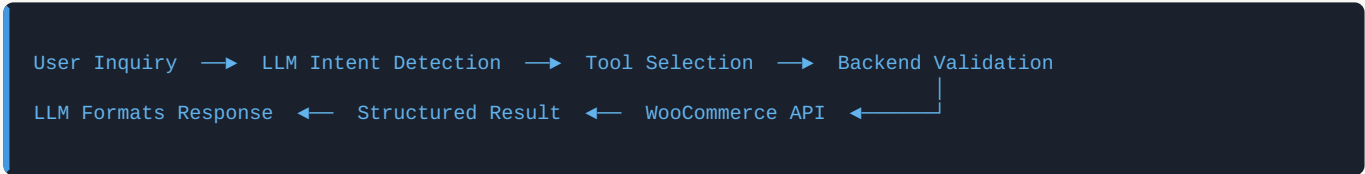
8. WordPress & WooCommerce Integration

For WordPress and WooCommerce sites, the integration engine uses direct REST API channels alongside crawling for rich data extractions:

- **WordPress Core:** Ingests posts, pages, categories, and custom post types via WP REST API. Private or user data is strictly isolated.
- **WooCommerce Commerce Layer:** Synchronizes products, categories, SKU data, pricing, stock availability, attributes, variations, images, and permalinks.

9. Action-Based AI & Commerce Engine

To prevent hallucinations and safety risks, the LLM is never given direct, raw access to external APIs or database execution. Instead, the assistant operates using strictly validated, predefined tools:



Configured tools include: `search_products()`, `get_product()`, `add_to_cart()`, `get_cart()`, `remove_from_cart()`, `update_cart()`, and `search_orders()`.

Interactive Product Cards

When product queries are resolved, the widget renders custom visual card components directly inside the conversation stream containing product images, pricing, stock indicators, a "View Product" direct link, and an "Add to Cart" action button.

10. WhatsApp Handoff & Escalation System

For the MVP phase, WhatsApp integration uses a lightweight, highly reliable click-to-chat redirection model. The escalation is triggered when:

- The visitor explicitly requests human support or inputs terms like "agent" or "representative".
- The RAG retrieval confidence score falls below strict thresholds or fails to find context chunks.
- Repeated unanswered questions or explicit user frustration signals are detected.

The system generates a direct, pre-populated WhatsApp URL launching the business owner's official support line, laying the groundwork for full WhatsApp Business API integration in later updates.

11. Multi-Tenant Architecture & Permissions

Multi-tenancy is enforced at every database and API layer using strict organizational hierarchy:

```

User Account
├── Organization
│   ├── Members & Roles (OWNER, ADMIN, MANAGER, AGENT, VIEWER)
│   └── Websites
│       ├── Knowledge Base & Vectors
│       ├── Conversations & Visitors
│       ├── Integrations (WooCommerce / Shopify)
│       ├── Customization & Branding Settings
│       └── Real-Time Analytics

```

12. Master Project Development Phases

PHASE 0 — Architecture & Foundation

Repository setup, Docker containerization, PostgreSQL + pgvector, Redis, FastAPI shell, Next.js dashboard shell, environment configs, base migrations, health checks, testing suite.

PHASE 1 — Authentication & Multi-Tenant SaaS Core

User registration, secure JWT authentication, password hashing, organizations, user-organization memberships, RBAC system, protected routes, tenant isolation middleware and tests.

PHASE 2 — Website Management

Website CRUD endpoints, unique site/widget ID generation, domain validation, platform detection architecture, site settings, installation instructions page, website dashboard layout.

PHASE 3 — Website Crawling & Content Discovery

Celery crawler workers, SSRF protection, robots.txt & sitemap parser, domain boundary checks, duplicate prevention, HTML text extraction, job tracking UI, page ingestion pipeline.

PHASE 4 — Knowledge Base & RAG Engine

Text cleaning, chunking strategies, local embedding generation, pgvector storage, semantic vector retrieval engine, context builder, source attribution metadata.

PHASE 5 — Local AI & Ollama Integration

Ollama service abstraction, local LLM orchestration, provider interface, prompt engineering, strict anti-hallucination rules, backend stream handling.

PHASE 6 — Embeddable Chat Widget

Lightweight JavaScript widget (widget.js), isolated shadow UI, floating launcher, chat interface, typing indicators, streaming responses, theme customization engine.

PHASE 7 — Conversation Management & History

Visitor sessions, message persistence, conversation status lifecycle (ACTIVE, WAITING, RESOLVED, ESCALATED), dashboard conversation manager, search and filters.

PHASE 8 — WordPress REST Integration

WordPress API connector, automated post/page/category extraction, incremental content sync.

PHASE 9 — WooCommerce Commerce AI Engine

CommerceProvider interface, WooCommerce provider, search/cart action tools, AI tool call validator, product card widget rendering, cart session management.

PHASE 10 — White-Label & Team Management

Custom branding settings, primary/secondary colors, logo uploads, team invitations, permission enforcement, website-level authorization.

PHASE 11 — WhatsApp Handoff Engine

Escalation rules, click-to-chat generator, automated handoff triggers, escalation analytics.

PHASE 12 — Shopify Integration

ShopifyProvider implementation matching CommerceProvider interface without modifying core AI logic.

PHASE 13 — Platform Analytics Engine

Conversation stats, AI resolution rates, unanswered query logs, WhatsApp click tracking, product interaction analytics, conversion events.

PHASE 14 — External AI Provider Expansion

OpenAI, Gemini, and Anthropic provider implementations integrated into the provider-agnostic core.

PHASE 15 — Monetization & Subscription Billing

SaaS plan tiers (Free, Starter, Growth, Agency), quota limits, Stripe/billing integration.

13. Master AI Coding Agent Prompts

To ensure high quality, developers should provide coding agents with the master system prompt first, followed strictly by individual phase prompts sequentially.

MASTER AI CODING AGENT PROMPT

You are the lead software architect and senior full-stack engineer responsible for building a production-ready, multi-tenant SaaS platform called AI Customer & Commerce Assistant.

The platform is an embeddable AI chatbot and customer-support/commerce assistant for business websites. The product must be designed as a serious SaaS application, not a prototype or demo.

1. PRODUCT OBJECTIVE

Build a SaaS platform where a business owner can: Create an account → Create an organization/workspace → Add multiple websites → Enter a website URL → Automatically discover the website's robots.txt and sitemap → Crawl and index the website → Build a searchable knowledge base → Run an AI chatbot using a LOCAL AI model → Embed the chatbot into WordPress, WooCommerce, Shopify, Next.js, or other websites using a JavaScript widget → Customize the chatbot appearance → Manage conversations → Add team members → Use WooCommerce product search and cart actions → Escalate conversations to WhatsApp → View analytics → Synchronize website content automatically.

The initial version MUST NOT require OpenAI, Gemini, Anthropic, or any paid external AI API. The initial AI implementation must use a local model through Ollama. However, the AI architecture must be provider-agnostic so external providers can be added later without rewriting the core application.

2. TECHNOLOGY STACK

Frontend: Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, React Hook Form, Zod, Recharts.

Backend: Python, FastAPI, Pydantic, SQLAlchemy, Alembic, Celery, Redis.

Database: PostgreSQL with pgvector extension.

AI: Ollama, Local LLM, Local embedding model. Create an abstraction: AIProvider with OllamaProvider (and future OpenAIProvider, AnthropicProvider, GeminiProvider).

3. ARCHITECTURAL PRINCIPLES

Follow Clean Architecture, SOLID principles, separation of concerns, dependency injection, repository/service patterns, strongly typed interfaces, secure multi-tenancy, and observability. Do not create huge single files or place business logic in API route handlers or database queries in UI components.

4. MULTI-TENANCY & AUTHENTICATION

Hierarchy: User → Organization → Websites. Multi-tenant isolation must be enforced at the database level for every tenant-owned record. Roles: OWNER, ADMIN, MANAGER, AGENT, VIEWER.

5. CRAWLER & RAG PIPELINE

Crawler must handle URL normalization, SSRF protection, robots.txt compliance, sitemap discovery, controlled depth limits, text cleaning, chunking, and pgvector embeddings storage. RAG pipeline must restrict responses to retrieved context and strictly prevent hallucinations.

6. FIRST TASK

Before writing application code: Analyze the repository, create architecture docs in /docs, set up initial directory structure, configure Docker, PostgreSQL + pgvector, Redis, FastAPI, Next.js, Alembic migrations, testing framework, .env.example, health endpoints, and README. Do NOT implement the chatbot yet.

PHASE 1 CODING AGENT PROMPT

Implement Phase 1 of the AI Customer & Commerce Assistant SaaS. Do not implement later phases.

Implement: User registration, Login, Logout, Secure password hashing, Authentication/session system, Protected routes, User model, Organization model, Organization membership, Roles and permissions, Multi-tenant authorization, Dashboard shell.

Users must be able to create an organization after registration. An organization can contain multiple members. Roles: OWNER, ADMIN, MANAGER, AGENT, VIEWER. All queries must enforce tenant isolation. Create database migrations, backend tests (especially tenant isolation), and frontend pages. Run tests before finishing.

PHASE 2 CODING AGENT PROMPT

Implement Phase 2 of the AI Customer & Commerce Assistant SaaS. Do not implement crawler or AI yet.

Implement: Website creation, edit, deletion, website list for authenticated organization, website detail view, website dashboard, unique public site/widget identifier generation, URL validation, domain normalization, platform detection architecture, site settings, installation instructions page. Create website, website_settings, and website_domains models, migrations, endpoints under /api/v1/websites, frontend routes, and tests.

PHASE 3 CODING AGENT PROMPT

Implement Phase 3: Website Crawling and Content Discovery.

Build background crawling system using Python workers and Celery/Redis. Implement: URL normalization, robots.txt parsing, sitemap.xml & sitemap_index.xml processing, domain-bound crawling, duplicate detection, HTML cleaning, page content extraction, job status tracking UI, crawl_jobs, crawl_pages, and knowledge_documents tables.

Security is critical: enforce SSRF protection blocking local/private IPs and internal metadata endpoints. Create tests for robots parsing, sitemap parsing, SSRF prevention, and crawl limits.

14. Primary Validation User Journey

The complete MVP value proposition is validated through a single, continuous user journey:

```
Business Owner → Sign Up → Create Organization → Add Website URL (https://example.com)
|
Widget Installed ← Copy Script ← Customize Widget ← Automatic Crawl & Sitemap Ingestion
|
Visitor Inquires → RAG Retrieval → Local AI Responds → Knowledge Base in pgvector
|
Product Query → WooCommerce Tool Search → Display Product Cards → Add to Cart
|
Low Confidence / Unresolved → WhatsApp Escalation Direct Handoff
```